Segundo Parcial
Practica 01



MEDICIÓN DE TIEMPO DE EJECUCIÓN CON FLUJOS CON Y SIN BUFFER

REPORTE

CONCURRENCIA Y PARALELISMO

INGENIERIA EN TECNOLOGIA
DE SOFTWARE

ALAN CAMPOS SUAREZ

UNIVERSIDAD AUTONOMA DE CAMPECHE

TABLA 1M

Resultados n = 1,000,000

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	1,000,000	223.424	211.169	202.611	212.40
BufferedWriter	1,000,000	81.666	96.993	74.467	84.38
FileReader	1,000,000	432.453	409.797	406.956	416.40
BufferedReader	1,000,000	55.861	58.142	50.346	54.78
FileOutputStream	1,000,000	5182.442	5065.374	5096.679	5114.83
BufferedOutputStream	1,000,000	80.165	82.761	77.679	80.20
FileInputStream	1,000,000	13308.580	13191.752	12771.835	13090.72
BufferedInputStream	1,000,000	160.217	169.464	160.144	163.27

TABLA 10M

Resultados n = 10,000,000

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	10,000,000	2247.460	2388.792	3042.879	2559.71
BufferedWriter	10,000,000	571.096	752.047	942.555	755.23
FileReader	10,000,000	4721.780	4229.300	4132.184	4361.09
BufferedReader	10,000,000	485.907	404.528	391.114	427.18
FileOutputStream	10,000,000	60028.758	56876.131	51893.273	56266.05
BufferedOutputStream	10,000,000	1824.295	673.491	664.744	1054.18
FileInputStream	10,000,000	193518.419	142275.299	141480.859	159091.53
BufferedInputStream	10,000,000	1852.411	1804.614	1821.322	1826.12

TABLA 100K

Resultados n = 100,000

Flujo utilizado	Tamaño	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	100,000	52.764	49.316	52.633	51.57
BufferedWriter	100,000	19.574	15.786	13.885	16.42
FileReader	100,000	65.370	63.078	64.319	64.26
BufferedReader	100,000	10.763	9.694	13.145	11.20
FileOutputStream	100,000	370.501	373.756	419.279	387.85
BufferedOutputStream	100,000	15.177	13.221	17.528	15.31
FileInputStream	100,000	1128.457	1156.687	1146.233	1143.79
BufferedInputStream	100,000	16.100	15.951	19.579	17.21

PREGUNTAS DE ANALISIS

1. ¿Qué diferencia observaste entre usar flujos con buffer y sus equivalentes directos? ¿Por qué ocurre?

La diferencia fue abismal. Los flujos con buffer fueron hasta 80 veces más rápidos. Por ejemplo, con 10 millones de datos, FileInputStream tardó un promedio de 159 segundos (2.6 minutos), mientras que BufferedInputStream tardó solo 1.8 segundos. Esto ocurre porque los flujos sin buffer hacen una llamada al sistema operativo (syscall) por cada byte o carácter, lo cual es lentísimo. Los flujos con buffer acumulan los datos en memoria RAM (en bloques de 8 KB) y los escriben/leen de golpe, reduciendo drásticamente los accesos al disco duro.

PREGUNTAS DE ANALISIS

2. ¿Cómo varía el tiempo de ejecución al aumentar n? ¿La relación es lineal?

El tiempo aumenta de forma aproximadamente lineal. Al multiplicar por 10 el tamaño de n (de 100k a 1M, y de 1M a 10M), el tiempo de ejecución también se multiplicó por 10 en la mayoría de los casos. Por ejemplo, FileWriter tardó 51 ms con 100k, 212 ms con 1M y 2559 ms con 10M. Se comporta igual en lectura que en escritura, ya que la cantidad de operaciones de E/S es directamente proporcional a la cantidad de datos.

PREGUNTAS DE ANALISIS

3. ¿Existe diferencia de rendimiento entre trabajar con caracteres (texto) y con bytes (binario)?

Sí, hay una ligera diferencia. En general, los flujos de texto (FileWriter/FileReader) son un poco más lentos que los binarios porque tienen que codificar y decodificar los caracteres a bytes usando el charset del sistema (como UTF-8). Sin embargo, la diferencia más grande y notable no es texto vs binario, sino el uso o no de buffer. Un flujo binario sin buffer es mucho más lento que un flujo de texto con buffer.

PREGUNTAS DE ANALISIS

4. ¿Qué tamaño tiene el buffer por defecto? ¿Puedes cambiarlo?

El tamaño por defecto es de 8192 bytes (8 KB). Sí se puede cambiar utilizando constructores sobrecargados. Por ejemplo: new BufferedWriter(new FileWriter("archivo.txt"), 16384); crearía un buffer de 16 KB. Aumentar el buffer puede mejorar el rendimiento en archivos muy grandes, pero consume más memoria.

PREGUNTAS DE ANALISIS

5. ¿Por qué BufferedReader ofrece readLine() y BufferedInputStream no?

Porque BufferedReader está diseñado para leer **texto** (caracteres), y entiende conceptos como los saltos de línea (\n, \r\n) y la codificación. Por eso puede acumular caracteres hasta encontrar un salto de línea y devolver un String. BufferedInputStream, por otro lado, trabaja con **bytes crudos** (datos binarios); no sabe qué es una línea de texto, simplemente lee números del 0 al 255.

CONCLUSION

En esta práctica pude comprobar de manera práctica la inmensa importancia de usar buffers en Java. Los resultados hablan por sí solos: intentar leer o escribir un archivo grande byte a byte (sin buffer) puede congelar el programa durante minutos, ya que obliga al disco duro a trabajar a su máxima capacidad de forma ineficiente. Al envolver los flujos con Buffered..., el rendimiento mejora exponencialmente porque se aprovecha la velocidad de la memoria RAM. A partir de ahora, siempre usaré BufferedWriter y BufferedReader para texto, y BufferedOutputStream y BufferedInputStream para binario, ya que es una buena práctica de programación indispensable para cualquier aplicación que maneje datos.